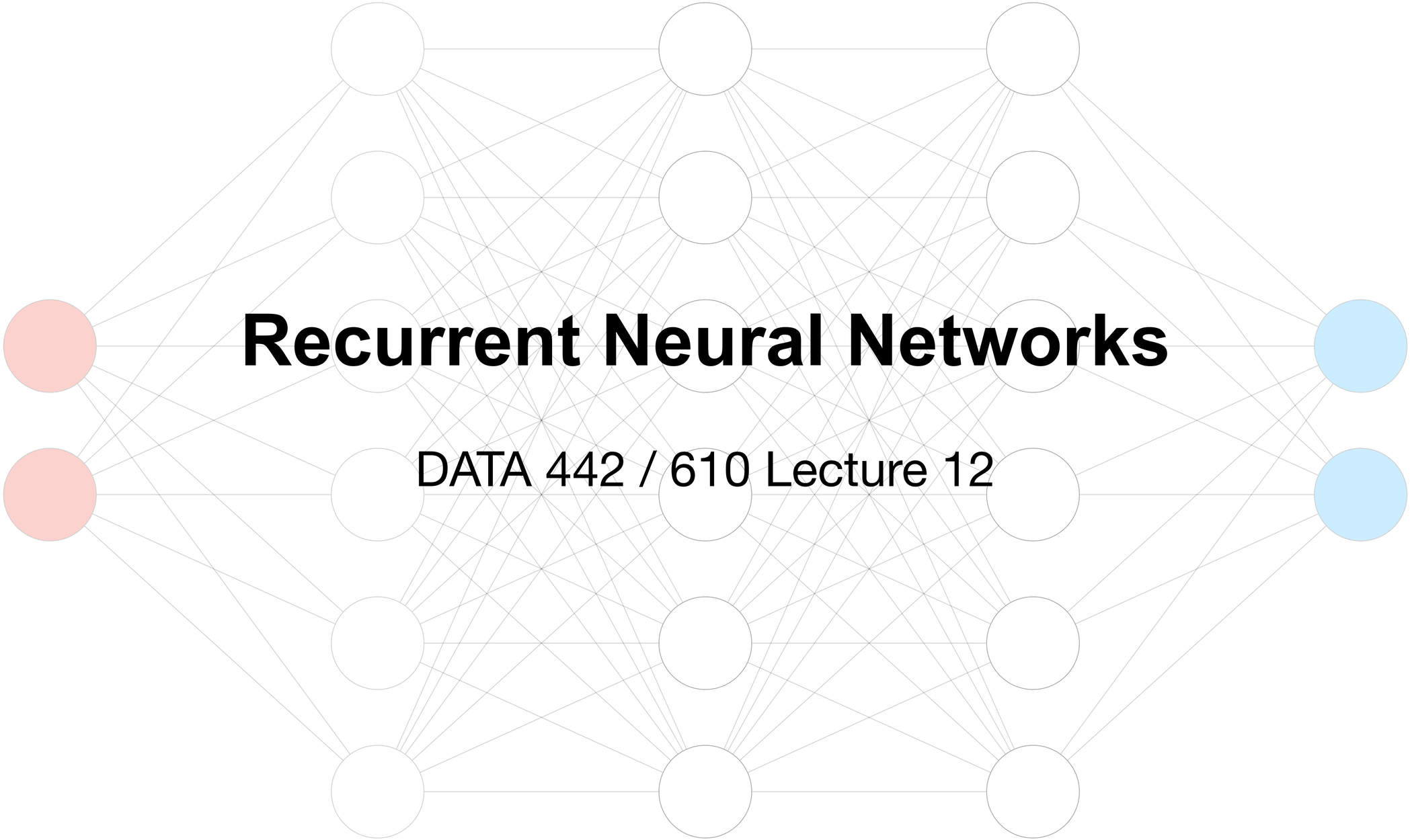


Recurrent Neural Networks

DATA 442 / 610 Lecture 12

Outline

- Review CNN Topics
- Encoder/Decoder networks
- Introduction to RNNs

Reading: *Deep Learning*, Chapter 10

Training deep (convolutional) neural networks

Deeper neural networks are more difficult to train

Batch normalization standardizes each layer's inputs to address *internal covariate shift*

Residual connections skip layers, improving training and making it easier to learn *identity* mappings

ArXiv:1502.03167

Input: Values of x over a mini-batch: $\mathcal{B} = \{x_{1...m}\}$;
Parameters to be learned: γ, β
Output: $\{y_i = \mathrm{BN}_{\gamma,\beta}(x_i)\}$

$$\mu_{\mathcal{B}} \leftarrow \frac{1}{m}\sum_{i=1}^{m} x_i \qquad \text{// mini-batch mean}$$

$$\sigma^2_{\mathcal{B}} \leftarrow \frac{1}{m}\sum_{i=1}^{m}(x_i - \mu_{\mathcal{B}})^2 \qquad \text{// mini-batch variance}$$

$$\widehat{x}_i \leftarrow \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma^2_{\mathcal{B}} + \epsilon}} \qquad \text{// normalize}$$

$$y_i \leftarrow \gamma\widehat{x}_i + \beta \equiv \mathrm{BN}_{\gamma,\beta}(x_i) \qquad \text{// scale and shift}$$

Algorithm 1: Batch Normalizing Transform, applied to activation x over a mini-batch.

ArXiv:1512.03385

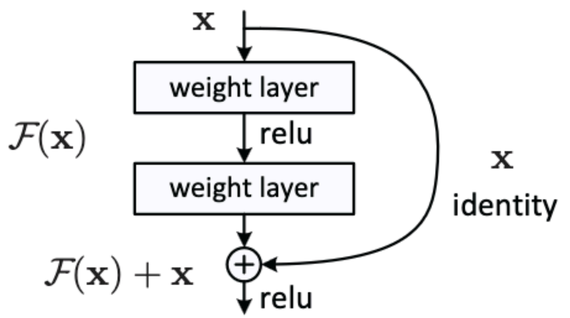


Figure 2. Residual learning: a building block.

A bit more on ResNet

Different residual block architectures have been proposed

A **bottleneck block** reduces the number of feature maps in each *spatial* (3x3) convolution

This allows deeper models with efficient memory footprints

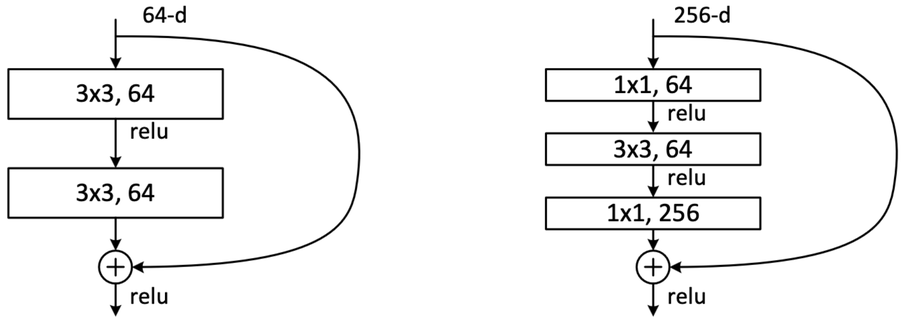


Figure 5. A deeper residual function $\mathcal{F}$ for ImageNet. Left: a building block (on 56×56 feature maps) as in Fig. 3 for ResNet-34. Right: a "bottleneck" building block for ResNet-50/101/152.

A bit more on TorchVision

Pre-trained models

Table of all available classification weights

Accuracies are reported on ImageNet-1K using single crops:

Weight
AlexNet_Weights.IMAGENET1K_V1
ConvNeXt_Base_Weights.IMAGENET1K_V1
ConvNeXt_Large_Weights.IMAGENET1K_V1
ConvNeXt_Small_Weights.IMAGENET1K_V1
ConvNeXt_Tiny_Weights.IMAGENET1K_V1
DenseNet121_Weights.IMAGENET1K_V1
DenseNet161_Weights.IMAGENET1K_V1
DenseNet169_Weights.IMAGENET1K_V1
DenseNet201_Weights.IMAGENET1K_V1
EfficientNet_B0_Weights.IMAGENET1K_V1
EfficientNet_B1_Weights.IMAGENET1K_V1
EfficientNet_B1_Weights.IMAGENET1K_V2
EfficientNet_B2_Weights.IMAGENET1K_V1
EfficientNet_B3_Weights.IMAGENET1K_V1
EfficientNet_B4_Weights.IMAGENET1K_V1
EfficientNet_B5_Weights.IMAGENET1K_V1
EfficientNet_B6_Weights.IMAGENET1K_V1
EfficientNet_B7_Weights.IMAGENET1K_V1
EfficientNet_V2_L_Weights.IMAGENET1K_V1
EfficientNet_V2_M_Weights.IMAGENET1K_V1
EfficientNet_V2_S_Weights.IMAGENET1K_V1
GoogLeNet_Weights.IMAGENET1K_V1
Inception_V3_Weights.IMAGENET1K_V1
MNASNet0_5_Weights.IMAGENET1K_V1
MNASNet0_75_Weights.IMAGENET1K_V1
MNASNet1_0_Weights.IMAGENET1K_V1

https://docs.pytorch.org/vis

Training recipes

Image classification reference training scripts

This folder contains reference training scripts for image classification. They serve as a log of how to train specific models, as provide baseline training and evaluation scripts to quickly bootstrap research.

Except otherwise noted, all models have been trained on 8x V100 GPUs with the following parameters:

Parameter	value
`--batch_size`	`32`
`--epochs`	`90`
`--lr`	`0.1`
`--momentum`	`0.9`
`--wd`, `--weight-decay`	`1e-4`
`--lr-step-size`	`30`
`--lr-gamma`	`0.1`

AlexNet and VGG

Since `AlexNet` and the original `VGG` architectures do not include batch normalization, the default initial learning rate `--lr 0.1` is too high.

```
torchrun --nproc_per_node=8 train.py\
    --model $MODEL --lr 1e-2
```

Here `$MODEL` is one of `alexnet`, `vgg11`, `vgg13`, `vgg16` or `vgg19`. Note that `vgg11_bn`, `vgg13_bn`, `vgg16_bn`, and `vgg19_bn` include batch normalization and thus are trained with the default parameters.

GoogLeNet

The weights of the GoogLeNet model are ported from the original paper rather than trained from scratch.

Inception V3

The weights of the Inception V3 model are ported from the original paper rather than trained from scratch.

Since it expects tensors with a size of N x 3 x 299 x 299, to validate the model use the following command:

```
torchrun --nproc_per_node=8 train.py --model inception_v3\
```

https://github.com/pytorch/vision/tree/main/references/classification

Code implementations

```
ock, Bottleneck]],

= False,

,
ion: Optional[list[bool]] = None,
able[..., nn.Module]] = None,

Norm2d
ayer

lation is None:
 tuple indicates if we should replace
 a dilated convolution instead
ilation = [False, False, False]
h_dilation) != 3:

ith_dilation should be None "
tuple, got {replace_stride_with_dilation}"

er_group
 self.inplanes, kernel_size=7, stride=2, padding=3,

f.inplanes)
ce=True)
2d(kernel_size=3, stride=2, padding=1)
layer(block, 64, layers[0])
layer(block, 128, layers[1], stride=2,
on[0])
layer(block, 256, layers[2], stride=2,
on[1])
layer(block, 512, layers[3], stride=2,
on[2])
eAvgPool2d((1, 1))
 block.expansion, num_classes)
```

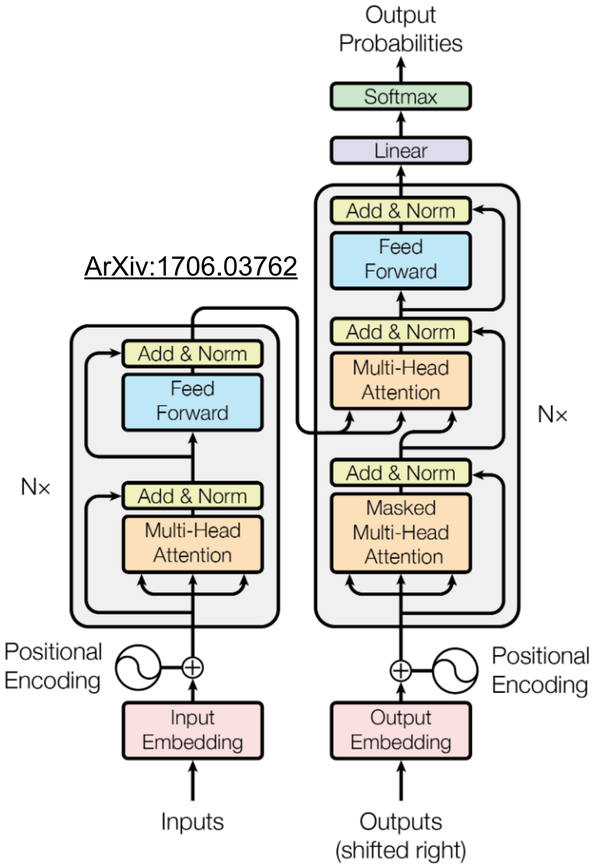
https://github.com/pytorch/vision/blob/main/torchvision/models/resnet.py

Towards Transformers

ChatGPT

Transformer components:

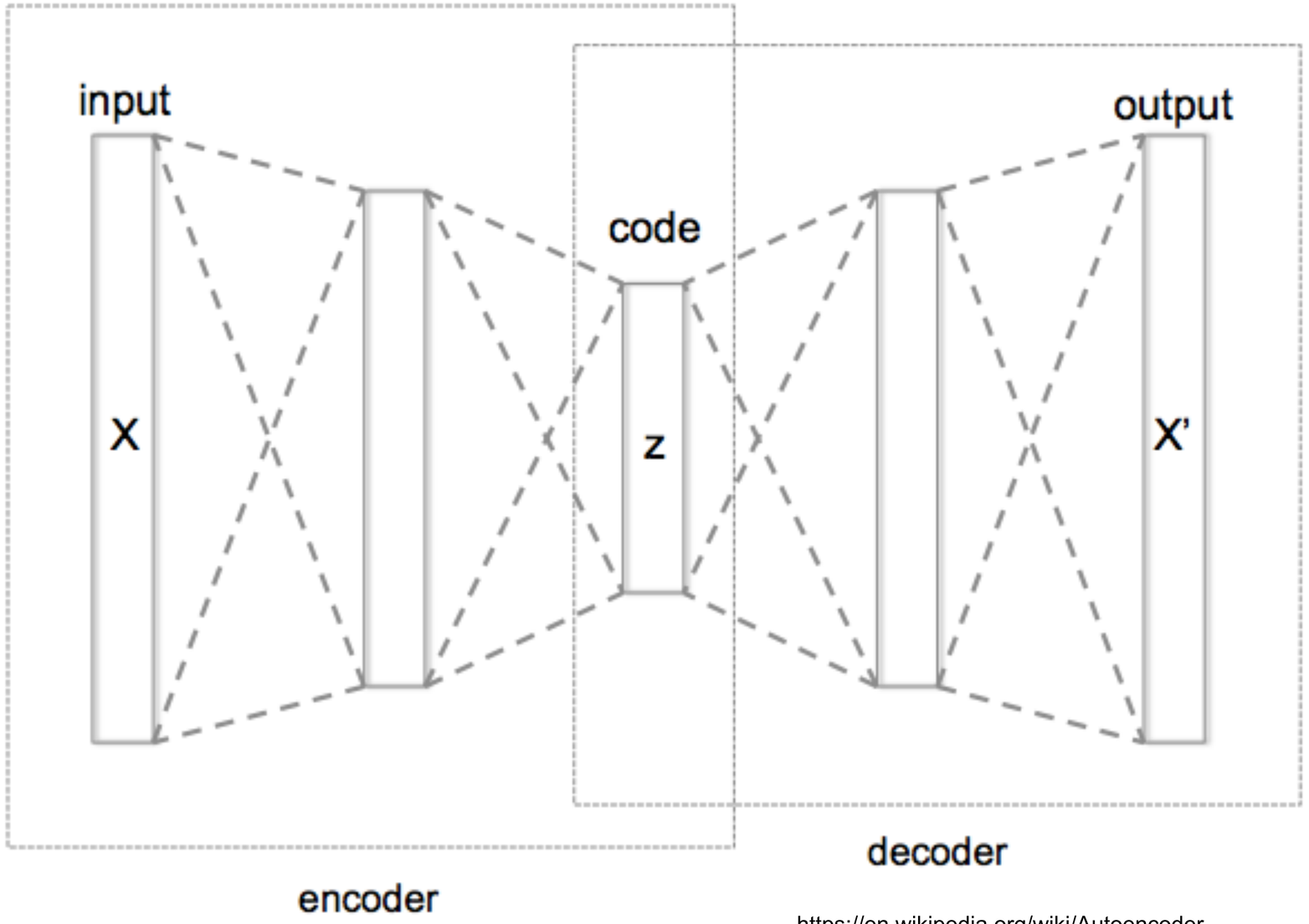
- Tokenization
- Embedding
- Encoder + decoder
- Attention mechanism



ArXiv:1706.03762

Figure 1: The Transformer - model architecture.

Encoder/Decoder networks



https://en.wikipedia.org/wiki/Autoencoder

Hands-on with Encoder/Decoder networks



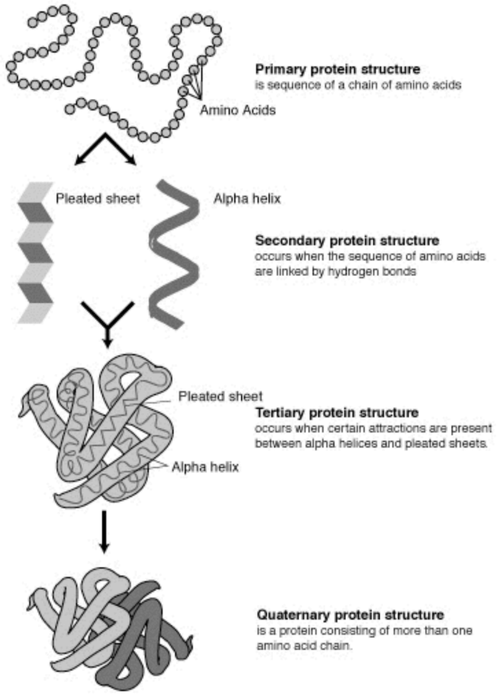
Sequential data

Sequential data appears in a particular order and can have arbitrary length

$$\{x^{(1)}, x^{(2)}, x^{(3)}, \ldots, x^{(t)}\} \quad t > 1$$

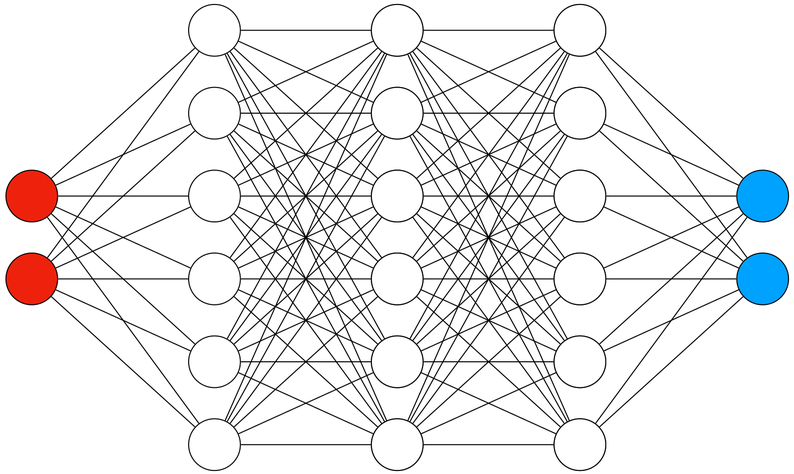
Examples:

- Time series data, e.g. stock prices each day, hour, minute
- Text, e.g. “Once upon a time there was a …”
- DNA sequences, e.g. ACCTGCTGATTACA
- Amino acid chains (AlphaFold)



How to process sequential data?

Fully-connected NN

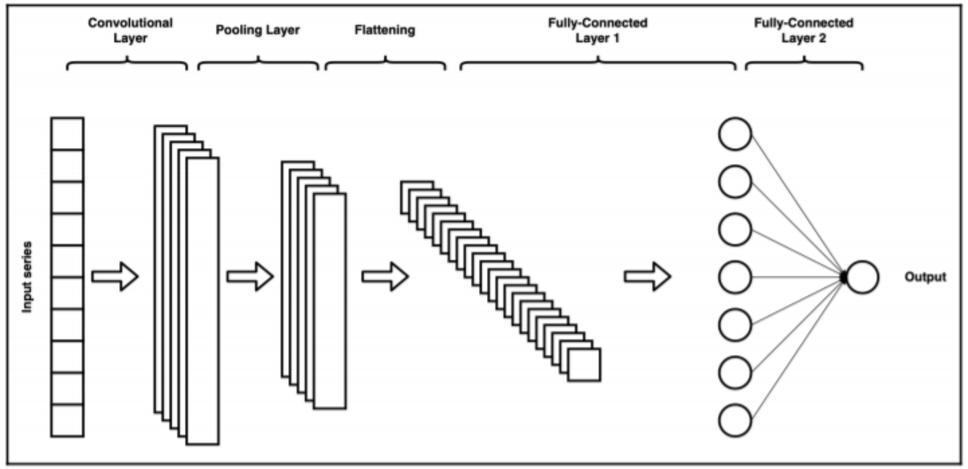


Requires fixed-length inputs

Cannot generalize to longer sequences

Struggles with variable ordering for some problems

1-dimensional CNN



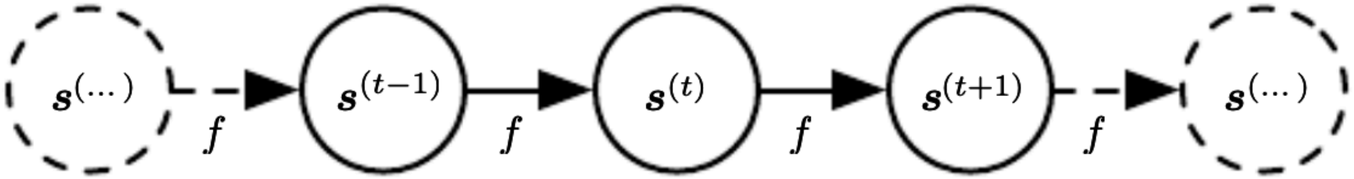
Accepts arbitrary sequence lengths due to **parameter sharing in time**

Shallow - only a few inputs contribute to predictions at each step

Fixed-size computational graph

Recursive structure in dynamical (sequential) systems

$$s^{(t)} = f\left(s^{(t-1)}\right)$$



$$s^{(1)} = f\left(s^{(0)}\right)$$

$$s^{(2)} = f\left(s^{(1)}\right) \qquad s^{(2)} = f\left(f\left(s^{(0)}\right)\right)$$

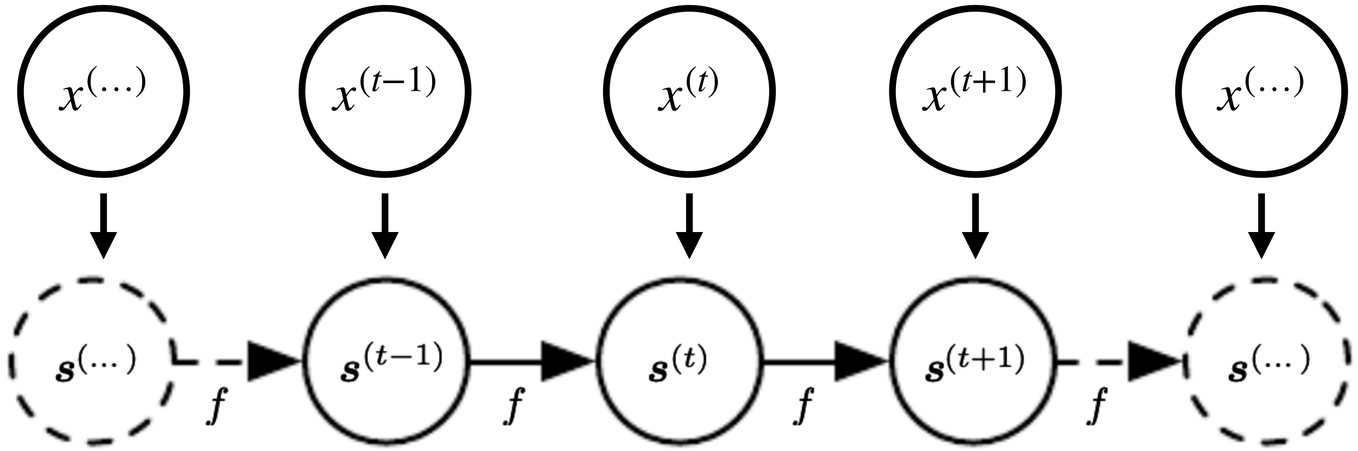
$$s^{(3)} = f\left(s^{(2)}\right) \qquad s^{(3)} = f\left(f\left(s^{(1)}\right)\right) \qquad s^{(3)} = f\left(f\left(f\left(s^{(0)}\right)\right)\right)$$

$$s^{(4)} = f\left(s^{(3)}\right) \qquad s^{(4)} = f\left(f\left(s^{(2)}\right)\right) \qquad s^{(3)} = f\left(f\left(f\left(s^{(1)}\right)\right)\right) \qquad s^{(4)} = f\left(f\left(f\left(f\left(s^{(0)}\right)\right)\right)\right)$$

Recursive structure can be “unfolded” into explicit computations

Dynamical systems with external driving

$$s^{(t)} = f\left(s^{(t-1)}; x^{(t)}\right)$$



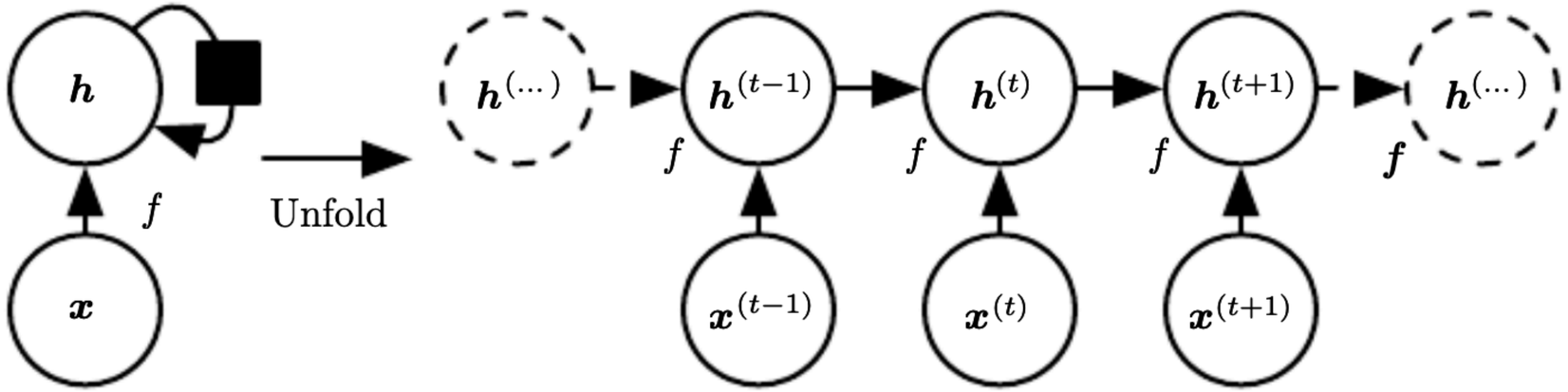
$$s^{(1)} = f\left(s^{(0)}; x^{(1)}\right)$$

$$s^{(2)} = f\left(s^{(1)}; x^{(2)}\right) \qquad s^{(2)} = f\left(f\left(s^{(0)}; x^{(1)}\right); x^{(2)}\right)$$

$$s^{(3)} = f\left(s^{(2)}; x^{(3)}\right) \qquad s^{(2)} = f\left(f\left(s^{(1)}; x^{(2)}\right); x^{(3)}\right) \qquad s^{(3)} = f\left(f\left(f\left(s^{(0)}; x^{(1)}\right); x^{(2)}\right); x^{(3)}\right)$$

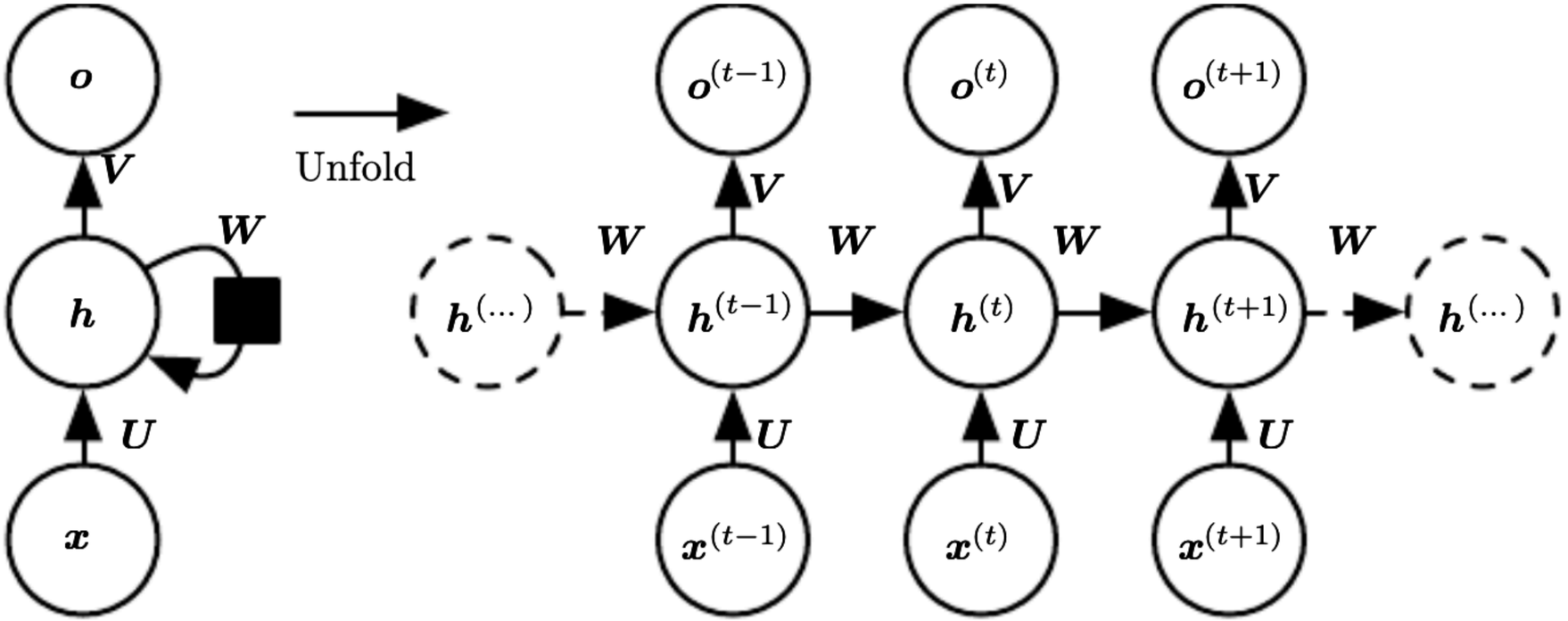
Recurrent neural networks

$$h^{(t)} = f\left(h^{(t-1)}; x^{(t)}; \theta\right)$$



RNN models evolution of a **hidden state** influenced by an input sequence

Recurrent neural networks



Recurrent neural networks share weights applied to both input sequence elements and the previous **hidden state**

Typical RNN formulations

Sentiment analysis

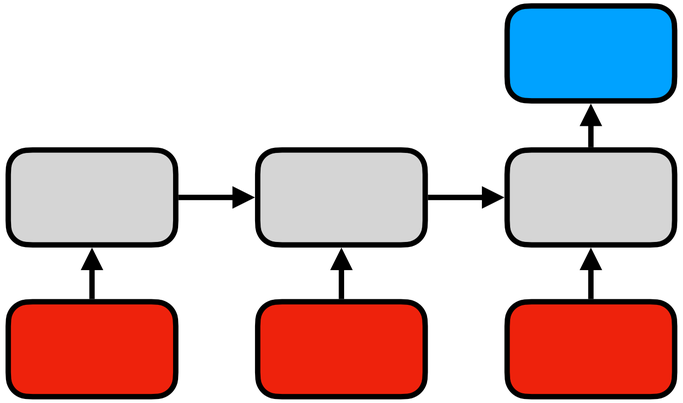
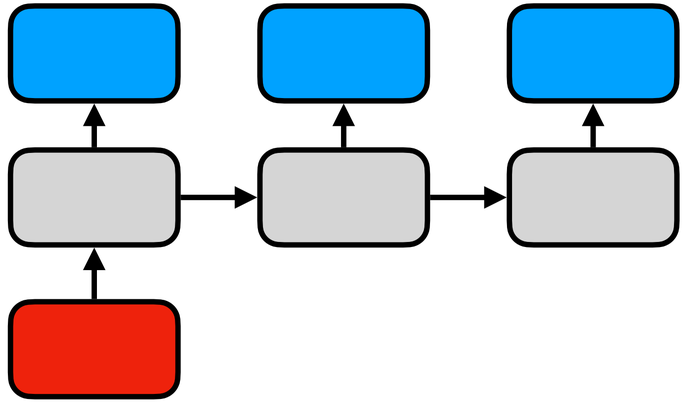
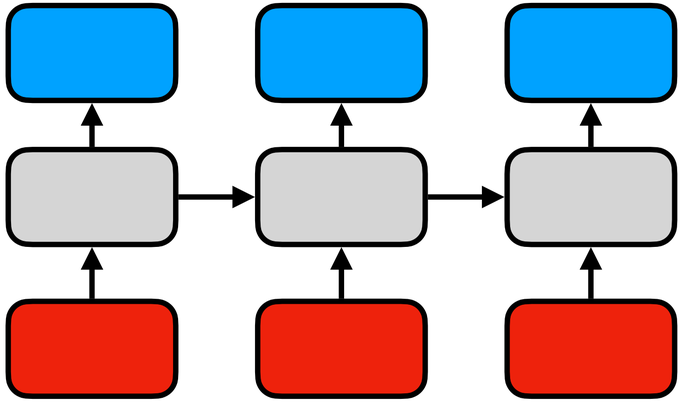


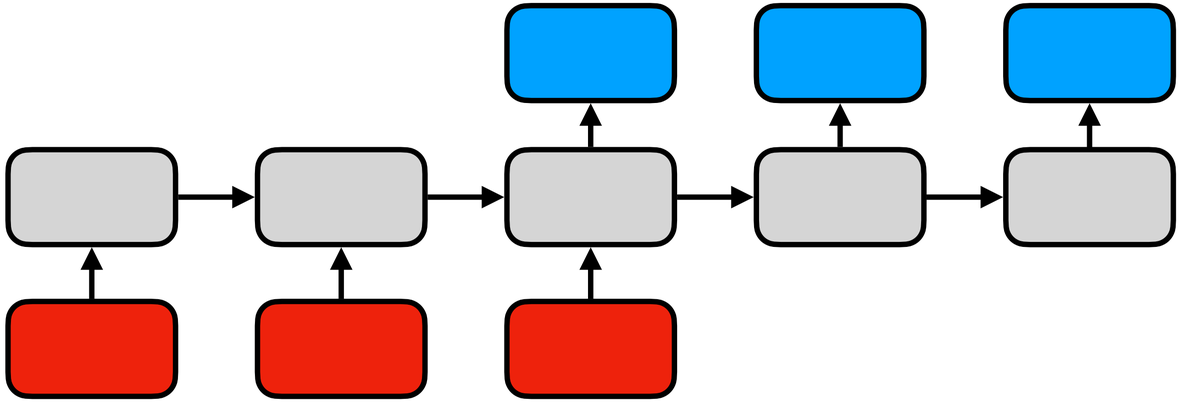
Image captioning



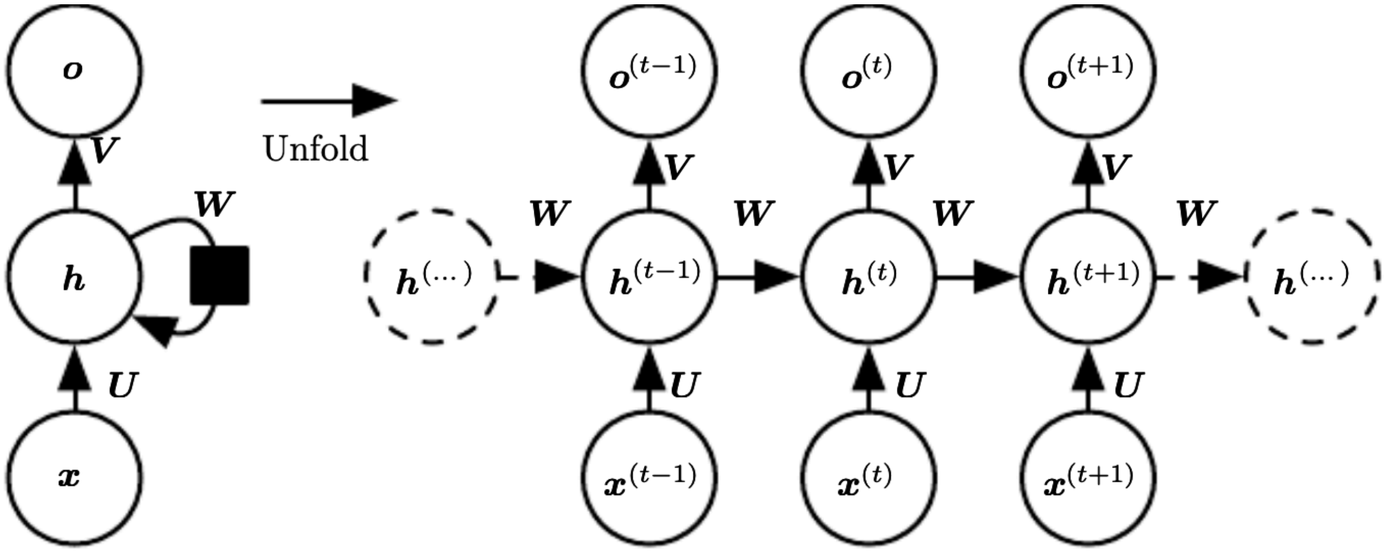
Video classification



Machine translation, text continuation



RNN forward pass



(Ignore W, V, U I do not like this notation)

$W_{ab} = W_{a \to b}$ = weight matrix for element a influencing element b

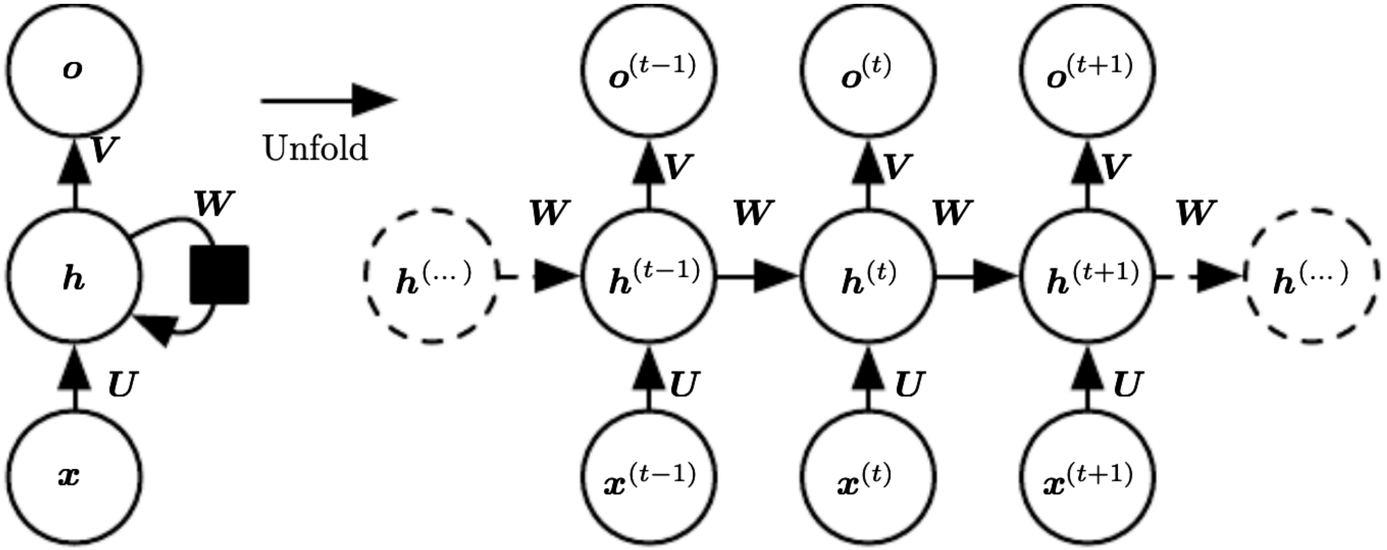
Preactivation: $z^{(t)} = b_h + W_{hh}\, z^{(t-1)} + W_{xh}\, x^{(t)}$

Hidden state: $h^{(t)} = \tanh z^{(t)}$

Output: $b_o + W_{ho} h^{(t)}$

(Optional) $y^{(t)} = \sigma(o^{(t)})$ or other activation, or softmax

RNN backward pass



Just propagate backward through time, right?

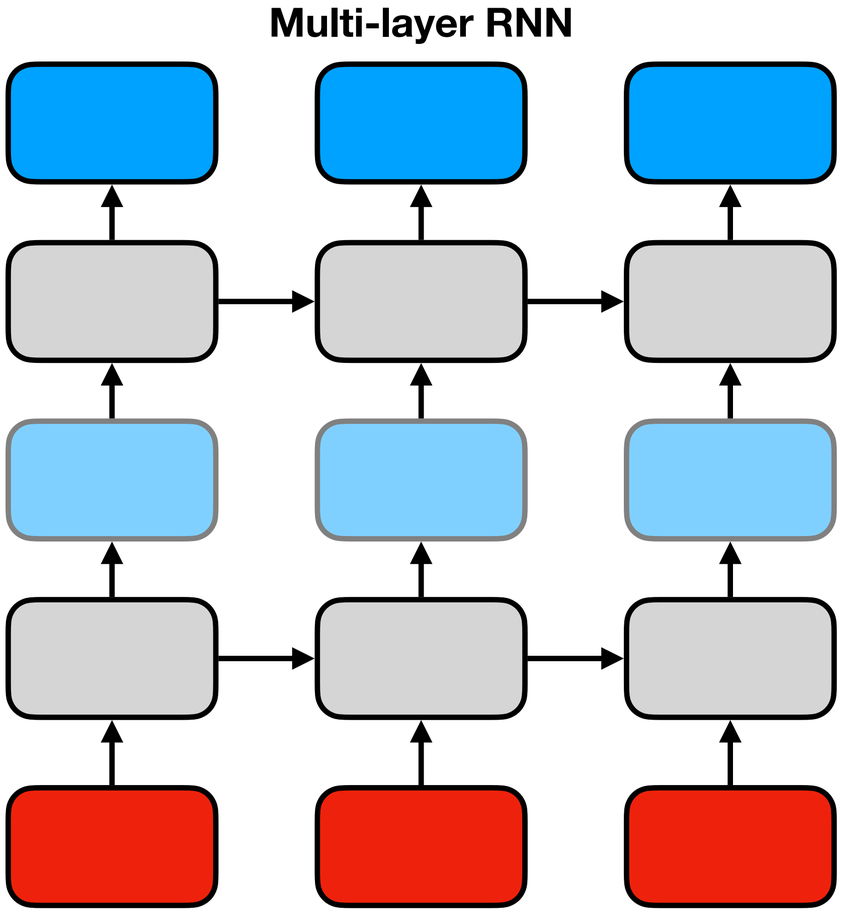
Because parameters are shared across time steps, gradients receive many contributions

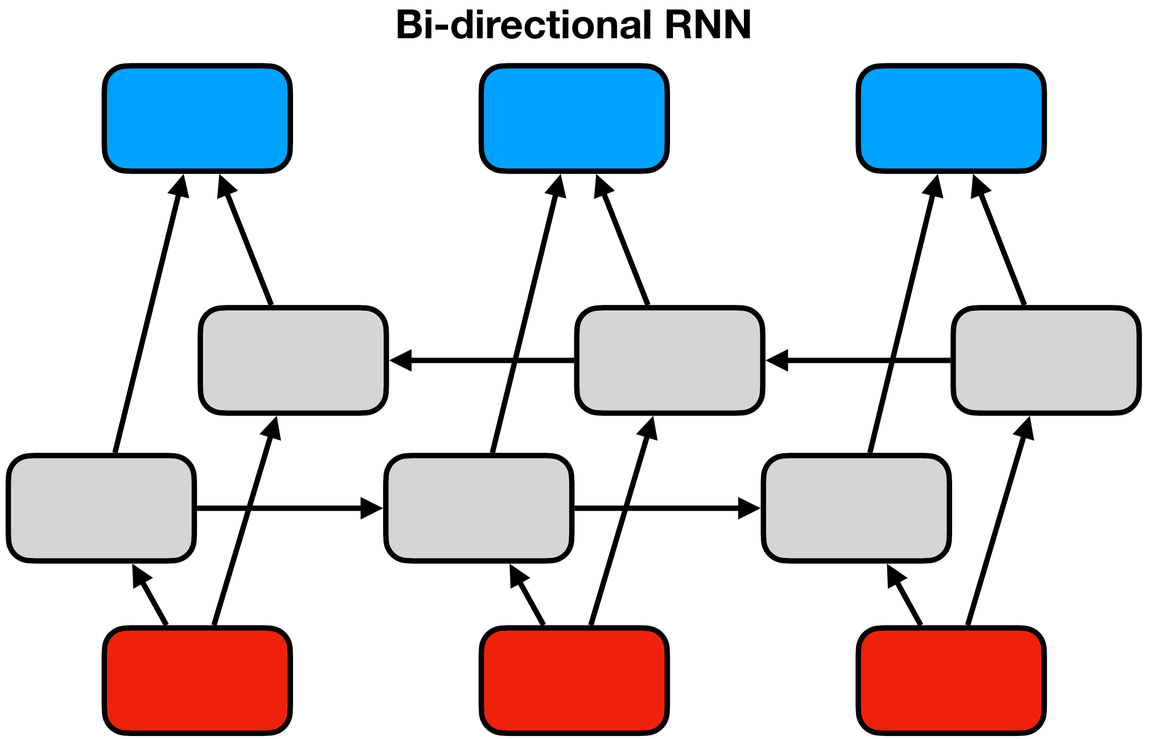
$$\frac{\partial L}{\partial W_{hh}} = \sum_t \frac{\partial L}{\partial h^{(t)}} \frac{\partial h^{(t)}}{\partial W_{hh}}$$

$$\frac{\partial L}{\partial h^{(t)}} = \frac{\partial L}{\partial h^{(t+1)}} \frac{\partial h^{(t+1)}}{\partial h^{(t)}} + \frac{\partial L}{\partial o^{(t)}} \frac{\partial o^{(t)}}{\partial h^{(t)}}$$

RNN training is ***very*** expensive…

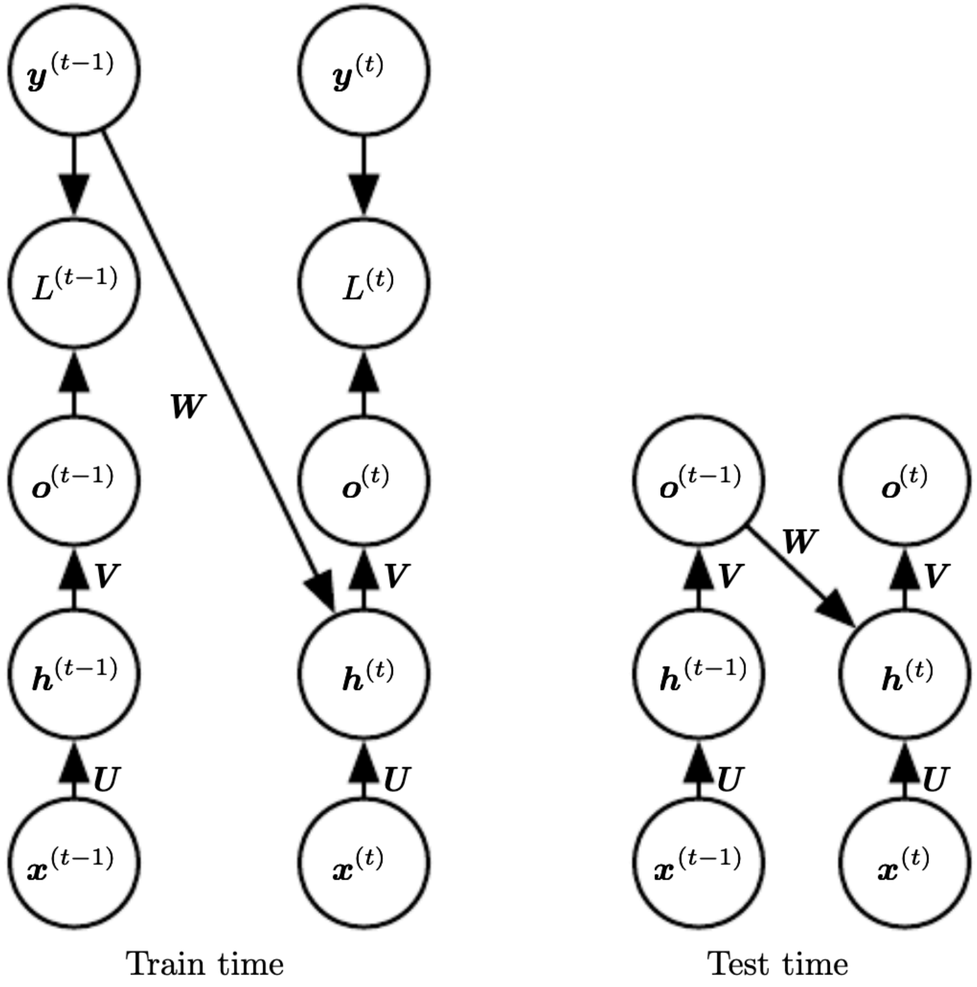
More RNN formulations





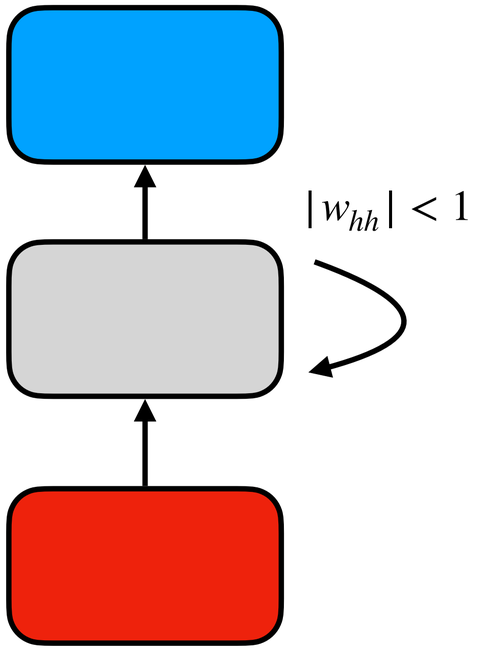
Teacher forcing: output-to-hidden recurrence

- Hidden-to-hidden training is slow because hidden states must be computed sequentially
- Output-to-hidden uses ground-truth values during training and model outputs during inference
- Gradients at each time step can be computed in parallel

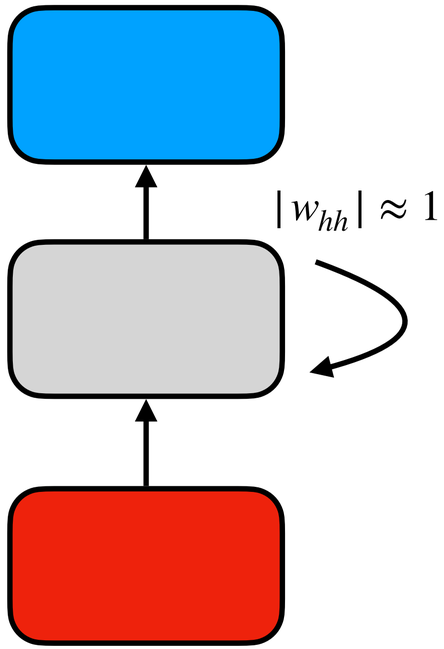


Challenges with long sequences

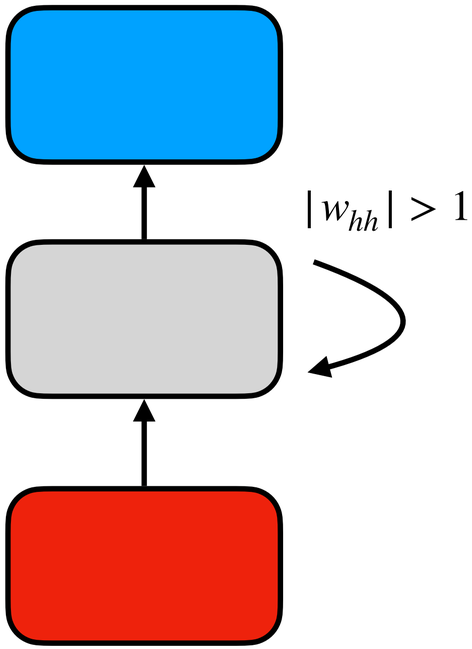
Vanishing gradient



Goldilocks zone



Exploding gradient



Strategies to handle vanishing/exploding gradients

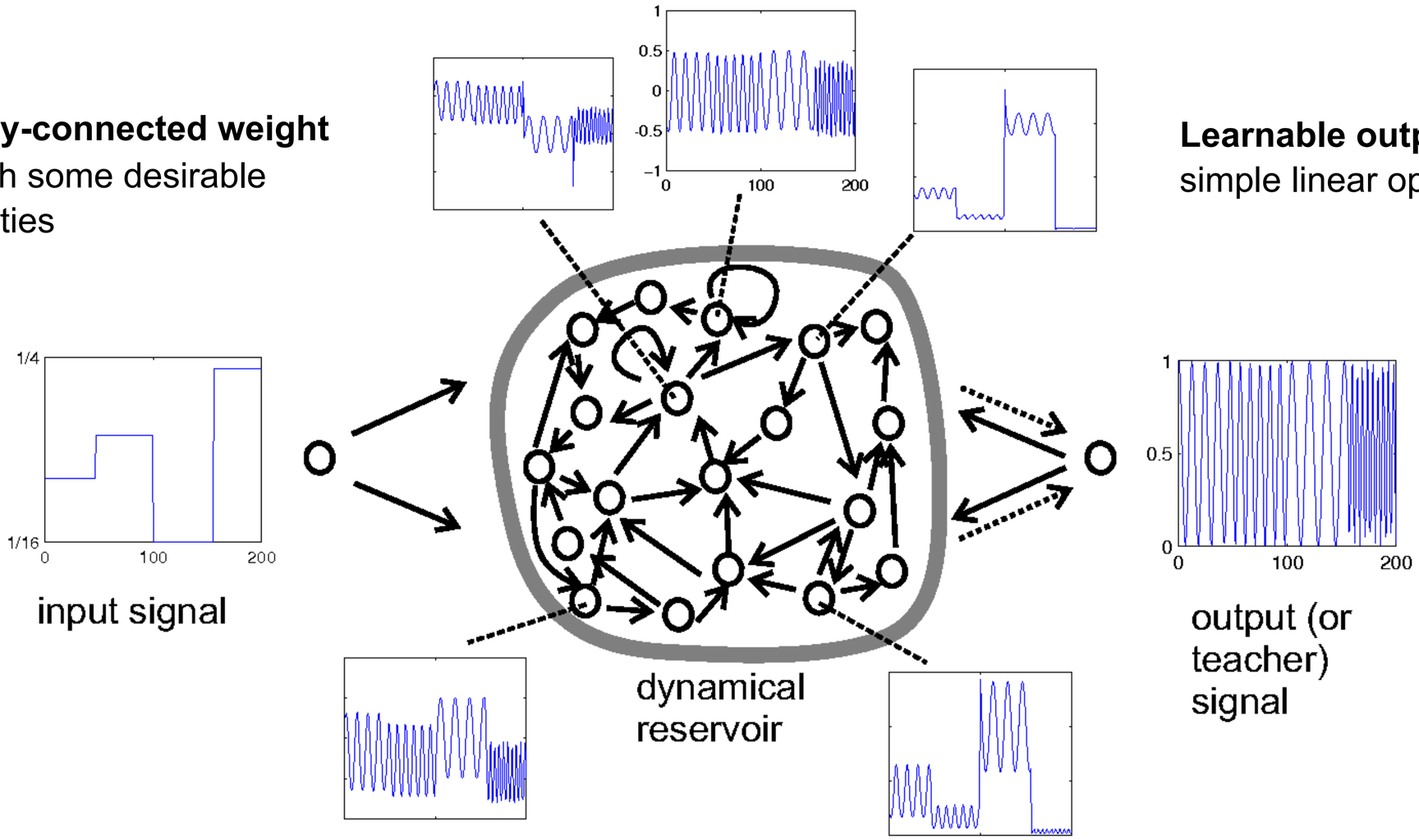
Gradient clipping: `torch.nn.utils.clip_grad_norm_(model.parameters(), max_gradient_value)`

Truncated Backpropagation through time: don’t backprop through the whole sequence

Long short-term memory (LSTM): gated connections allow the *model* to select whether information is propagated through time

Reservoir computing and Echo State Networks

Fixed, sparsely-connected weight matrix W_{hh} with some desirable spectral properties



Learnable output matrix W_{oh} via simple linear optimization

https://en.wikipedia.org/wiki/Echo_state_network

That’s all!

- Slides will be posted on the course website: https://colen-teaching.github.io/DATA442-Fall-2026
- Lab demos will also be posted
- ***PS2*** due October 23
- **REMEMBER:** TA Office Hours Tuesday 12-1pm, Student Success Center